KAIROS-6G: KPI-Aware Intelligent Reasoning for Operations and SUNRISE-6G

1st Paulito P. Palmes
AI for IT Automation
IBM
Dublin, Ireland
paulpalmes@ie.ibm.com

2nd Christos Verikoukis
ISI/ATH and University of Patras
Patras, Greece
verikoukis@athenarc.gr

3rd Kostas Ramantas
R+D Department
Iquadrat Informática
Barcelona, Spain
kramantas@iquadrat.com

***Abstract*—Federated 6G experimentation produces heterogeneous workflows, telemetry streams, model artifacts, and operational logs that are difficult to interpret during live trials and after experiment completion. This paper presents KAIROS-6G (KPI-Aware Intelligent Reasoning for Operations and SUNRISE-6G), an explainability layer for SUNRISE-6G that turns intent-driven MLOps into evidence-grounded operational reasoning. KAIROS-6G integrates Argo Workflows, *AutoMLPipeline*, *AutoAD*, MLflow, OpenTelemetry/Prometheus KPI streams, LLM reasoning, and Model Context Protocol (MCP) tool access in an operator-facing web command deck. Its core contribution is a closed loop from operator intent to workflow execution, KPI anomaly detection, experiment-memory capture, and human-readable explanation. *AutoAD* converts noisy KPI time series into detector-supported anomaly events, MLflow preserves run parameters and artifacts, and MCP gives the LLM scoped access to workflow, telemetry, and experiment evidence. The resulting explanations correlate KPI deviations with workflow stages, logs, detector settings, and model outcomes while preserving trust boundaries through read-only evidence gathering, redaction, source attribution, and approval-gated mutations. KAIROS-6G therefore supports sustainable SUNRISE-6G operations by making federated experiments more inspectable, repeatable, and reusable across testbeds and vertical use cases. giving the agent unrestricted control.**

***Index Terms*—LLM/MCP evidence gathering, KPI anomaly detection, explainable operations, time-series monitoring, MLOps**

I. Motivation

SUNRISE-6G [1] targets repeatable 6G experimentation across federated testbeds, service APIs, and native AI functions. MLOps practice already emphasizes workflow orchestration, reproducibility, metadata tracking, monitoring, and feedback loops [2]. In federated trials, however, operational evidence is spread across workflow engines, telemetry stores, experiment trackers, object stores, and logs. A latency spike or packet loss event is hard to interpret unless it can be aligned with workflow stages, detector settings, model outputs, and infrastructure state.

KAIROS-6G addresses this gap by adding a KPI-aware reasoning loop around existing MLOps components. Operators state an intent such as running anomaly detection over a KPI stream or asking why a metric changed. The system triggers the appropriate workflow, collects KPI and experiment evidence, and returns a concise explanation with supporting facts, missing evidence, and next checks. The goal is not to make an autonomous operator. The goal is to make each experiment inspectable, repeatable, and reusable after the live trial ends.

II. Architecture

KAIROS-6G combines four planes. The interaction plane is a browser command deck with panels for workflow templates, parameters, logs, Prometheus plots, anomaly streams, and agent prompts. This keeps intent close to evidence instead of scattering an operator across dashboards.

The execution plane uses Argo Workflows [3] and *AutoMLPipeline* [4], [5]. Reusable AutoML and anomaly-detection components are packaged as parameterized workflow templates. Argo records inputs, stage history, logs, retries, suspension, and resubmission state, making each intent replayable and auditable. MLflow [6] stores run parameters, metrics, tags, artifacts, models, and plots.

The observability plane uses OpenTelemetry and Prometheus to store time-aligned infrastructure and service KPIs. KAIROS currently treats CPU, memory, network throughput, and latency as first-class metrics; the same pattern applies to 6G service indicators such as packet loss, handover delay, slice quality, and vertical-application probes.

The anomaly plane uses *AutoAD*. KPI streams are converted to tabular time-series inputs and processed by an ensemble of unsupervised detectors from pycaret and scikit-learn outlier-detection families [7], [8]. Supported families include PCA, KNN, Histogram, Isolation Forest, Elliptic Envelope, One-Class SVM, and Local Outlier Factor. The *votepercent* parameter controls the detector consensus needed to mark a sample anomalous. Low thresholds surface weak or early signals; higher thresholds focus on stronger spikes or change points. The output is a plot plus structured metadata: metric, query, time range, step size, detector mode, threshold, warnings, and anomaly timestamps.

The reasoning plane is evidence-first. MCP tools [9] expose scoped, structured access to Argo templates, workflow status, node status, logs, MLflow experiments, runs, metric history, and artifacts. The LLM [10] receives a deterministic evidence bundle rather than screenshots or unbounded dashboard text. Mutating actions such as submit, retry, stop, suspend, resume,

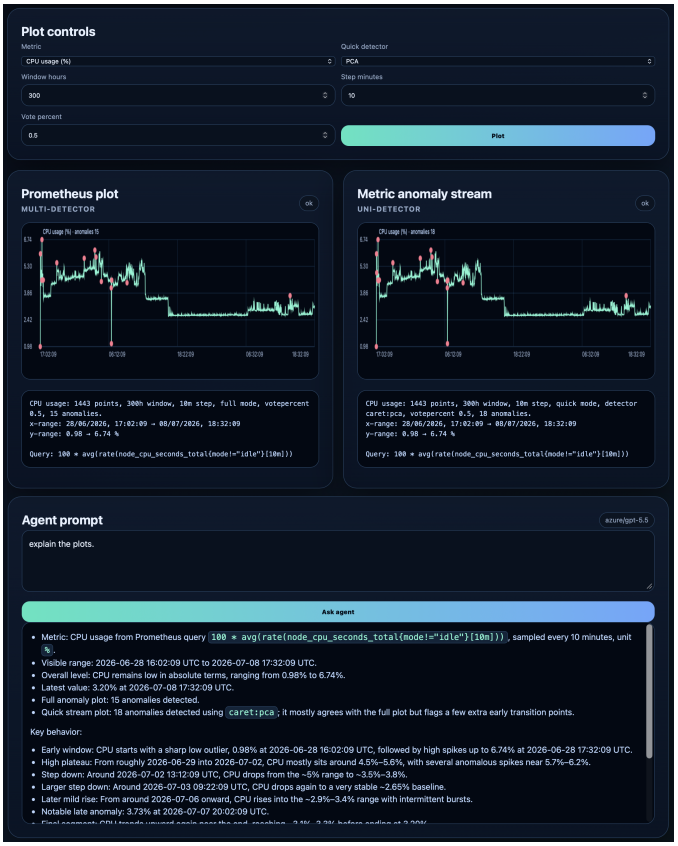


Fig. 1. KAIROS UI panels used in the agentic explanation loop: KPI plot controls define the metric, detector, time window, and vote threshold; the agent prompt panel turns selected evidence into an explanation request.

delete, or resubmit remain separate and require explicit permission.

III. Agentic KPI Explainability

KAIROS agents coordinate evidence; they do not replace Argo, Prometheus, MLflow, or *AutoAD*. A typical investigation starts with selected UI state: workflow, template, KPI, time range, anomaly marker, and detector settings. Fig. 1 shows these controls and prompt surfaces. The agent gathers workflow phase, active nodes, logs, MLflow run summaries, artifacts, metric history, and anomaly metadata before writing an explanation.

Three rules keep explanations auditable. First, evidence precedes reasoning: facts from Argo, Prometheus, *AutoAD*, and MLflow are gathered before narrative text is generated. Second, causal hypotheses require timestamp alignment between KPI anomalies and workflow stages, logs, resource changes, or model activity. Third, correlation is labeled as correlation. When evidence is incomplete, the output names the missing check instead of pretending to know root cause.

The explanation format is intentionally small. It reports observed anomaly, relevant workflow context, experiment context, likely explanation, counter-evidence or limits, and next action. Example next actions include inspecting one pod log, comparing a previous MLflow run, changing *votepercent*, or rerunning the same workflow with the same parameters. This turns noisy KPI plots into operational triage notes grounded in specific components.

IV. Sustainability Value

KAIROS-6G supports SUNRISE-6G sustainability by preserving operational memory. Each experiment can leave workflow parameters, stage history, logs, KPI traces, anomaly settings, MLflow artifacts, and explanation notes. New sites can reuse proven templates and detector settings; vertical owners can compare KPI behavior across trials; and facility operators can distinguish local site effects from recurring federation patterns.

Future work should add federation metadata to each evidence object: testbed identity, vertical use case, experiment identifier, tenant scope, workflow name, and KPI namespace. KAIROS should also move from single-run analysis to longitudinal knowledge extraction, where repeated anomalies, recurring workflow failures, and model-performance trends become reusable playbooks. Read-only investigation can be automated safely, while write actions should stay approval-gated. This gives SUNRISE-6G a path toward low-touch service assurance without removing human accountability.

Acknowledgment

This work was partly funded by the SUNRISE-6G project (Grant #: 101139257), co-funded by the European Union and Smart Networks and Services Joint Undertaking (SNS JU).

References

[1] SUNRISE-6G Consortium, “SUNRISE-6G Project,” https://sunrise6g.eu, 2024, accessed: 2026-07-08.

[2] D. Kreuzberger, N. Kühl, and S. Hirschl, “Machine learning operations (MLOps): Overview, definition, and architecture,” *IEEE Access*, vol. 11, pp. 31 866–31 879, 2023. [Online]. Available: https://doi.org/10.1109/ACCESS.2023.3262138

[3] Argo Project, “Argo Workflows,” https://github.com/argoproj/argo-workflows, 2024, accessed: 2026-07-08.

[4] P. Palmes. (2020, Aug.) Automlpipeline.jl: a package that makes it trivial to create complex ml pipeline structures using simple expressions. [Online]. Available: https://doi.org/10.5281/zenodo.3980593

[5] P. P. Palmes, A. Kishimoto, R. Marinescu, P. Ram, and E. Daly, “Designing machine learning pipeline toolkit for automl surrogate modeling optimization,” *Proceedings of the JuliaCon Conferences*, vol. 1, no. 1, p. 129, 2023. [Online]. Available: https://doi.org/10.21105/jcon.00129

[6] MLflow Project, “MLflow,” https://github.com/mlflow/mlflow, 2024, accessed: 2026-07-08.

[7] PyCaret Contributors, “PyCaret Anomaly Detection API,” https://pycaret.readthedocs.io/en/latest/api/anomaly.html, 2024, accessed: 2026-07-08.

[8] scikit-learn Developers, “Novelty and Outlier Detection,” https://scikit-learn.org/stable/modules/outlier_detection.html, 2024, accessed: 2026-07-08.

[9] Model Context Protocol Contributors, “Model Context Protocol Documentation,” https://modelcontextprotocol.io, 2024, accessed: 2026-07-08.

[10] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, and A. Mian, “A comprehensive overview of large language models,” *ACM Trans. Intell. Syst. Technol.*, Jun. 2025, just Accepted. [Online]. Available: https://doi.org/10.1145/3744746